# NOTAS DE AULA TEÓRICO-PRÁTICA

**Fundamentos de Recursão e Atividades de Programação**

**Disciplina:** Matemática Discreta / Algoritmos
**Curso:** Bacharelado em Sistemas de Informação
**Professor:** Eduardo Max Amaro Amaral
**Instituição:** IFES - Campus Colatina

## 1. O que é Recursividade?

A **recursão** é uma técnica fundamental em computação e matemática que define a solução de um problema em termos de uma ou mais versões menores desse mesmo problema. Em termos práticos de programação, diz-se que uma função é recursiva quando dentro de seu próprio corpo existe uma ou mais chamadas para si mesma.

**Analogia da Fila do Cinema:**
Imagine que você está em uma fila de cinema e deseja saber qual é a sua posição, mas não enxerga a bilheteria. Em vez de percorrer toda a fila, você pergunta para a pessoa à sua frente: *"Qual é o seu número?"*. Essa pessoa também não sabe e repassa a pergunta para quem está na frente dela. Isso se repete até a pergunta chegar na **primeira pessoa da fila**, que olha para a bilheteria e responde com certeza: *"Eu sou o número 1!"* (**Caso Base**). Ao receber a resposta, o segundo descobre que é o $1 + 1 = 2$, o terceiro descobre que é o $2 + 1 = 3$, até a informação retornar a você.

## 2. Estrutura de uma Função Recursiva

Toda rotina recursiva estruturada de forma correta deve conter obrigatoriamente dois elementos:

- **Caso Base (Solução Trivial ou Condição de Parada):** É a condição em que o resultado é obtido diretamente sem novas chamadas recursivas. Evita que o programa entre em um ciclo infinito e cause estouro de pilha (*stack overflow*).
- **Passo Recursivo (Chamada a Si Mesma com Redução):** A função processa parte dos dados e chama a si mesma passando um subproblema estritamente menor, garantindo convergência em direção ao caso base.

## 3. Exemplo Canônico: Cálculo do Fatorial

O fatorial de um número natural $n$ ($n!$) é definido naturalmente por recorrência:

$$n! = 1, \text{ se } n = 0 \text{ (Caso Base)} \quad | \quad n! = n \cdot (n-1)!, \text{ se } n > 0 \text{ (Passo Recursivo)}$$

### Rastreamento da Pilha de Execução (Call Stack) para n = 3

```
fatorial(3)
  ├──> aguarda: 3 * fatorial(2)
  │      ├──> aguarda: 2 * fatorial(1)
  │      │      ├──> aguarda: 1 * fatorial(0)
  │      │      │      └──> CASO BASE atingido: retorna 1
  │      │      └──> retorna: 1 * 1 = 1
  │      └──> retorna: 2 * 1 = 2
  └──> retorna: 3 * 2 = 6 (Resultado Final)
```

### Implementações do Fatorial nas Linguagens da Disciplina

**Linguagem C:**

```c
#include <stdio.h>

int fatorial(int n) {
    if (n <= 0) return 1;           // 1. Caso Base
    return n * fatorial(n - 1);    // 2. Passo Recursivo
}
```

**Python 3:**

```python
def fatorial(n):
    if n <= 0:
        return 1                  # 1. Caso Base
    return n * fatorial(n - 1)    # 2. Passo Recursivo
```

**GNU Octave / MATLAB:**

```
function res = fatorial(n)
    if n <= 0
```

```
        res = 1;                    % 1. Caso Base
    else
        res = n * fatorial(n - 1); % 2. Passo Recursivo
    endif
endfunction
```

## 4. Vantagens e Desvantagens da Recursão

| Critério | Recursão | Iteração (Laços for / while) |
|---|---|---|
| **Elegância e Clareza** | Código conciso, expressivo e próximo da formulação matemática formal do problema. | Pode exigir variáveis de controle e acumuladores auxiliares mais verbosos. |
| **Consumo de Memória** | Cada chamada aloca um novo registro de ativação na pilha de memória (*stack*). | Consumo constante de memória ($O(1)$), reaproveitando as variáveis do laço. |
| **Desempenho** | Possui custo de troca de contexto entre funções, sendo em geral mais lenta. | Execução direta no processador, geralmente mais rápida para operações lineares simples. |

## 5. Atividades Práticas de Programação (Para Entrega)

📋 **Diretrizes para Submissão:**

1. As questões podem ser resolvidas em **C**, **Python** ou **GNU Octave**, à escolha do aluno.
2. É **terminantemente proibido** o uso de laços de repetição ( `for` , `while` ou `do-while` ) dentro do cálculo principal; a solução deve ser puramente recursiva.
3. Identifique claramente em comentários no código a linha correspondente ao **Caso Base** e a linha do **Passo Recursivo**.

### EXERCÍCIO 1 — Potenciação Inteira Exponencial ($x^y$)

**Descrição:** Implemente a função recursiva `potencia(x, y)` que calcule o valor de uma base inteira $x$ elevada a um expoente inteiro $y \geq 0$, sabendo que $x^0 = 1$ e $x^y = x \cdot x^{y-1}$.

**Exemplos:** `potencia(2, 4) → 16` | `potencia(7, 0) → 1`

### EXERCÍCIO 2 — Soma dos Primeiros $N$ Inteiros Consecutivos

**Descrição:** Crie a função recursiva `soma_ate_n(n)` que receba um inteiro $n \geq 1$ e retorne a soma dos inteiros de $1$ até $n$, considerando a solução trivial $S(1) = 1$ e a chamada recursiva $S(n) = n + S(n - 1)$.

**Exemplos:** `soma_ate_n(5) → 15` | `soma_ate_n(1) → 1`

### EXERCÍCIO 3 — Contagem Regressiva com Efeito Colateral

**Descrição:** Escreva um procedimento recursivo `contagem_regressiva(n)` que receba um número inteiro positivo ou nulo $n \geq 0$ e imprima na saída os números de $n$ até $0$ em ordem decrescente, imprimindo a mensagem `"Decolagem!"` logo após o zero.

**Exemplo de Saída para n = 3:** `3` , `2` , `1` , `0` , `Decolagem!`

### EXERCÍCIO 4 — Sequência de Fibonacci (Recorrência de 2ª Ordem)

**Descrição:** Desenvolva a função recursiva `fibonacci(n)` que determine o $n$-ésimo termo da sequência de Fibonacci para $n \geq 1$, considerando os casos base $F(1) = 1$ e $F(2) = 1$, e para $n > 2$: $F(n) = F(n - 1) + F(n - 2)$.

**Exemplos:** `fibonacci(1) → 1` | `fibonacci(6) → 8` | `fibonacci(7) → 13`

**EXERCÍCIO 5** **Soma dos Dígitos de um Número Decimal**

**Descrição:** Construa a função recursiva `soma_digitos(n)` que receba um número inteiro positivo ***n*** e devolva a soma de todos os seus dígitos individuais. Utilize operações de divisão inteira por 10 (para reduzir o número) e resto de divisão por 10 (para extrair o último dígito).

**Exemplos:** `soma_digitos(1974) → 1 + 9 + 7 + 4 = 21` | `soma_digitos(8) → 8`